REAL-TIME GESTURE RECOGNITION

Using LSTM

By

AKHIL MATHEW CHERIAN

Submitted To

The University of Roehampton

In partial fulfilment of the requirements
for the degree of

MASTER OF SCIENCE IN DATA SCIENCE

Abstract

The purpose of this study was to use computer vision to recognize Sign Language in real time with high accuracy. The purpose of such a project is to bridge the gap between those who can hear well and those who are deaf or hard of hearing. This can be prevented by assembling a dataset of images corresponding to the alphabets, numbers, or signs utilized by deep neural networks. The name of the letter being signed is indicated on these photographs. After training on larger datasets with many more images and classifications, they are processed via a neural network using transfer learning to help the machine "learn" what is being signed.

Declaration

I hereby certify that this report constitutes my own work, that where the language of others is used, quotation marks so indicate, and that appropriate credit is given where I have used the language, ideas, expressions, or writings of others.

I declare that this report describes the original work that has not been previously presented for the award of any other degree of any other institution.

AKHIL MATHEW CHERIAN
Date: 06/09/2023

Acknowledgments

I want to extend my sincere gratitude to everyone who helped this project be completed successfully. With the help and support of numerous people and organizations, this project has been a journey of learning and development.

First, I thank the almighty lord for giving me confidence and putting me in the right direction throughout the project.

I want to express my profound gratitude to Changjiang He, who oversaw my project, for their essential mentoring, support, and direction during the entire endeavour. Their knowledge and opinions have been very helpful in forming this research.

I also want to express my gratitude to the University of Roehampton for providing the tools and resources I needed to complete this assignment successfully.

My sincere gratitude is extended to my family and friends for their support, tolerance, and patience throughout the duration of this effort. Your confidence in me has served as a constant inspiration.

Last but not least, I want to express my gratitude to all the participants who kindly offered their time and help with data collection; without them, this research would not have been possible.

Thank you all for being a part of this journey.

Table Of Contents

1. Introduction.....6
   1.1. Research Question or Problem Statement.....6
   1.2. Aims.....6
   1.3. Objectives.....7
   1.4. Legal, Social, Ethical, and Professional Considerations.....8
   1.5. Motivation and Background.....9
   1.6. Report Overview.....10
2. Literature or Technology Review.....11
3. Methodology.....14
4. Implementation..... 18
5. Results.....29
   5.1 Evaluation.....30
   5.2 Related Work.....31
6. Conclusion.....32
   6.1. Reflection.....32
   6.2. Future Work.....33
7. References.....34
8. Appendices..... 36

1. Introduction

Gesture recognition depends on a variety of parameters, including the kinematics of the hands and facial expressions. People who have hearing loss frequently use it to communicate with one another, but those who do not have it rarely do. People engage in fewer social interactions as a result of just speaking directly to others who also have hearing loss. Real-time interpretation is a possibility, although it is not always feasible and can be very expensive. Therefore, a machine translation system would be highly beneficial. Numerous unique solutions have lately been developed in this field. This model outlines a process for understanding and translating Custom Sign Language into regular text. I'll create a program in this project that translates sign language into OpenCV. It describes a method for comprehending and translating Custom Sign Language into conventional text.

1.1 PROBLEM STATEMENT

Persons who are unable to communicate employ hand gestures and hand signals. Ordinary people have difficulty understanding their language. As a result, a system is necessary that distinguishes numerous signs and gestures and sends information to regular people. It helps to bridge the gap between physically challenged people and the general society. The project's purpose is to develop a real-time sign language detection system using action recognition algorithms. The device will scan sign language gestures performed in front of a webcam and deliver immediate feedback to the user. The goal is to be able to accurately classify a wide range of sign language motions into known classifications, allowing for effective communication. The project will include gathering and preparing a representative dataset, training a deep-learning model for gesture recognition, and developing a user-friendly interface for real-time feedback. The goal is to create an accessible tool that facilitates communication for those who are deaf or hard of hearing while simultaneously encouraging inclusiveness and facilitating sign language interpretation in a range of contexts.

1.2 AIMS

The project attempts to create a real-time sign language detection system using action recognition algorithms. The primary goals are to reliably classify varied sign language motions into predefined classes, to offer users immediate feedback, and to promote accessibility and inclusion for those with hearing impairments. The project intends to improve communication by developing a deep

learning model, collecting a representative dataset, and designing an intuitive user interface, demonstrating the practical application of Artificial Intelligence (AI) in promoting inclusivity.

1.3 OBJECTIVE

- Real-Time Gesture Recognition: Implement a real-time sign language gesture detection system that accepts webcam footage, recognizes hand landmarks, and instantly interprets sign language gestures.
- Multi-class Gesture Recognition: Train the deep learning LSTM model to accurately identify a wide range of sign language movements such as 'hello,' 'thanks,' and 'yes,' among others.
- Data Collection and Preprocessing: Create a sample dataset of sign language motions by recording video sequences of different people executing various signs. For training, save the appropriate key points from the hand landmarks as numpy files.
- Model training and evaluation: To ensure correct gesture classification, train the LSTM model on the received key points data and verify its performance using metrics such as category accuracy.

- Threshold-based Recognition: Implement a threshold-based technique to determine when a sign gesture is detected confidently enough to eliminate false positives and enable reliable recognition.

- Real-Time Feedback and User Interface: Real-time display of recognized sign language gestures on the screen, giving users immediate feedback on detected signals. Create an easy-to-use interface for easy interaction.

- Continuous Gesture Detection: Make sure the system can recognize entire sign motions that may span multiple frames, allowing for continuous detection of sign expressions.

- Demonstration of AI Application: Demonstrate how AI and deep learning techniques may be utilized to understand sign language motions, proving the technology's practical applicability in assisting persons with hearing problems.

- Inclusivity and Accessibility: Promote inclusivity and accessibility by bridging the communication gap between hearing-impaired people and others.

1.4 LEGAL, ETHICAL, SOCIAL AND PROFESSIONAL CONSIDERATIONS

- Legal Considerations: Various legal aspects have been given careful consideration during the development of this project in order to ensure compliance with applicable rules and laws:
- Data Usage and Privacy: I have taken precautions to gather and use data sensibly in accordance with data protection laws. Privacy concerns have been addressed at every level of the project, and the people whose data is utilized have provided their approval.
- Intellectual property: All pre-existing models, libraries, or frameworks that I used in this project were used in line with the licenses that came with them. I have also considered employing appropriate intellectual property safeguards to protect our exclusive solutions.

- Regulations: Throughout the course of the project, I have been aware of and compliant with any laws or standards that may be relevant to the use of computer vision and artificial intelligence technology.
- Ethical Considerations: My effort has been developed with ethical considerations at its core in order to ensure fairness, transparency, and accountability:
- Fairness & Bias: I am committed to correcting any potential biases in my data in order to prevent biased forecasts. My dataset has been carefully chosen to reflect a diversity of populations in order to minimize bias in this system.
- Transparency: I'm committed to being upfront and truthful about how this AI system works. Users of this app will receive clear explanations of the judgments this AI model takes.
- Accountability: I am prepared to take ownership of any negative results or errors brought on by this AI system. Users will have access to resources for reporting issues and asking for assistance.
- Social Considerations: The larger societal repercussions of this initiative have been carefully explored in order to ensure that it is used ethically and broadly to benefit everyone:
- Impact on Society: I have assessed the project's potential for social impact in an effort to limit negative effects on different communities. The project's goal is to avoid making currently existing inequities or privacy concerns worse.
- Accessibility: I have made sure that everyone, including those with disabilities, can use this application by giving accessibility first attention. I'm committed to making sure that everyone can benefit from this innovation.
- Cultural Sensitivity: I'm conscious of the importance of differences in how gestures or actions are perceived depending on the culture. This software takes into account numerous cultural contexts and is sensitive to cultural variances.
- Professional Considerations: As I worked on this project, I tried to uphold professionalism and norms of conduct:
- This gesture detection technology was developed with great accuracy, especially for critical applications. The system's weaknesses have been made very evident.
- The success of this initiative depends on open and sincere dialogue with users, stakeholders, and the community. I'm committed to sharing information about changes, upgrades, and any issues associated with utilizing this program.

- Professionalism: Throughout the entire project, I adhered to best practices for coding and documentation.

1.5 MOTIVATION AND BACKGROUND

The goal of sign language recognition is the development of algorithms and strategies for precisely identifying and comprehending the provided symbol sequences. To correctly categorize a given signal from a group of likely indications, research has concentrated on discovering beneficial qualities and differentiating tactics, but sign language is more than just a collection of expressive hand gestures.

What is Gesture Recognition?

Gesture recognition is a branch of computer science and language technology that employs mathematical formulas to translate human touch. Computer vision is a subfield in computer science. Any movement or bodily position can result in a gesture, although the hands and the face are the two areas where they are most frequently used. The ability to recognize emotions through the touch of hands and faces is a current hot topic in the industry. Users can interact with or control objects without actually touching them by employing a simple touch. Using cameras and computer vision algorithms, several methods have been developed to translate the signal language.

What is Sign Language?

In sign languages, often known as sign language, visual signals are employed to convey meaning. Both sign language and non-sign language items are expressed using sign language. Sign languages are complete natural languages with their grammar and vocabulary. Even though there are some glaring parallels among sign languages, they are not often acknowledged or accepted. Speaking and signing are both regarded as natural forms of language by linguists, indicating that they both evolved through an unplanned, slow process over a long period of time. Although sign language shouldn't be confused with it, body language is a type of nonverbal communication.

1.6 REPORT REVIEW

There are several sections that are mentioned in this report. First is the Literature review or Technology review followed by the methodology and project management. Then after that, there is the implementation part in which I explain the workflow of the model with a diagram. Implementation is followed by results in which I show the real-time prediction of my model of the sign language and after that, there is a conclusion, references, and appendices.

2. Literature Or Technology Review

LITERATURE REVIEW

The paper [1] study recommended an innovative deep learning-based pipeline architecture using the capabilities of SSD, 2DCNN, 3DCNN, and LSTM for efficient automatic hand sign language identification from RGB videos. They projected the features utilizing the model onto three surfaces in order to input new hand skeleton features to the 3DCNNs and produce richer features. They also applied 3DCNNs to pixel-level and heat map features to extract discriminative features.

One of the challenges faced in the paper [2] with computer vision models, particularly for sign language, is real-time recognition. They provide a model for real-time isolated hand sign language recognition (IHSLR) using RGB video that primarily consists of a single-shot detector, a 2D convolutional neural network, singular value decomposition (SVD), and a huge short-term memory. They utilize an SVD technique to extract efficient, condensed, and discriminative features from the projected 3D hand key point coordinators.

The paper [3] describes a deep CNN architecture with five layers that can recognize and classify sign languages from photographs of hand gestures. During the training, validation, and blind testing phases, the suggested methodology uses both static (0-9 and A-Z) and dynamic (alone, afraid, furious, etc.) gestures to strengthen the system.

In the paper [4] a novel approach to human action recognition is presented. The evaluation of video content and feature extraction are the main objectives of this approach. The Gaussian Mixture Models (GMM) and KF algorithms are used to track human movements to provide motion features. Other features are based on all visual aspects of each frame on a video sequence using a recurrent neural network model with a gated recurrent unit. The main advantages of this novel approach are the analysis and feature extraction from each frame and second of the video.

In the paper [5] researchers looked at a number of conventional machine learning and deep learning models to categorize American sign language. Additionally, a robust, user-independent test phase and k-fold cross-validation were provided. In contrast to past efforts, this one involved user-independent testing and/or validation phases.

TECHNOLOGY REVIEW

The following technological decisions were taken for the project on gesture recognition:

1. Mediapipe: Google's MediaPipe is an excellent computer vision library with pre-trained models for hand landmark detection, location estimation, and face detection. In this study, MediaPipe is used to distinguish hand landmarks from a webcam video, which serves as the foundation for sign language recognition.
2. OpenCV: (Open-Source Computer Vision Library) is a popular open-source computer vision and image processing library. In this project, it is used to take video from a camera, analyze images, and present visualizations. OpenCV enhances the project's real-time capabilities by efficiently managing video streams.
3. Long Short-Term Memory (LSTM): An LSTM is a type of recurrent neural network (RNN) that works well with sequential input. LSTM is used in this study to analyze the temporal correlations in sign language gestures, allowing for accurate sign classification.
4. Numpy: Numpy is a fundamental Python library for numerical computation. It is utilized to handle and manipulate data in this study, such as storing key points gathered from hand landmarks and grouping them into sequences for training the LSTM model.
5. Matplotlib: It is a powerful plotting library that was utilized in the study to visualize the camera video, display recognized gestures, and plot performance metrics during model training. It makes it easier to offer visual feedback to users as well as analyze the model's training progress.
6. TensorBoard: TensorBoard is a TensorFlow visualization tool that may be used to monitor and debug deep learning models. It is used in the project to create visualizations of the LSTM model's architecture, performance metrics, and training progress. TensorBoard aids in the optimization of model hyperparameters and the convergence of models.
7. Python: Python is the primary programming language used in the project due to its simplicity, ease of use, and availability of numerous libraries for computer vision and deep learning. Python's environment and libraries facilitate rapid prototyping and development.

8. Gesture Recognition and Action Classification: The project uses action recognition techniques to classify sign language motions. The system recognizes sign expressions accurately and in real-time by combining hand landmark detection, sequential data processing, and deep learning.
9. Real-Time Interaction and User Interface: The project prioritizes providing real-time input to users through an intuitive user interface. The camera stream is displayed on the screen, together with identified movements, allowing for easy user interaction.
10. Accessibility and Inclusivity: The technology employed in this project is in line with the larger goal of creating solutions that are both accessible and inclusive. The device improves inclusivity by identifying sign language motions in real-time, allowing persons with hearing impairments to communicate effectively.

Overall, the technological research focuses on the application of computer vision, deep learning, and user interface design to create a viable and successful sign language recognition system. A strong library and framework combination allows for real-time recognition and facilitates communication for the deaf community.

Why did I choose these technologies?

The sign language recognition project's technology evaluation indicates a well-thought-out set of tools and libraries that contribute to the system's effective implementation. The project makes use of MediaPipe's ability to recognize hand landmarks and estimate pose, precisely collecting critical points from real-time hand movements.

OpenCV enhances the system's real-time capabilities by rapidly processing camera feeds, allowing image manipulation, and offering visualizations. NumPy's numerical processing capabilities handle and change data efficiently, ensuring smooth data organization for training the LSTM model.

The deep learning component is built around TensorFlow and Keras, which allow for the creation and training of the LSTM model for sign language gesture identification. TensorBoard extends TensorFlow by displaying visualizations of training progress and performance data, optimizing model hyperparameters, and ensuring model convergence.

The combination of these tools promotes efficient development, real-time performance, and a user-friendly gesture recognition interface. Furthermore, using Python as the primary programming language improves usability and access to a huge ecosystem of libraries for computer vision and deep learning. The technology review focuses on a well-rounded set

of tools that correspond with the project's goals, resulting in a comprehensive and impactful sign language recognition system.

3. Design Or Methodology

Different approaches have been tried to solve the problem of two-way communication between hearing and non-hearing people. A large number of cameras or heavy, difficult-to-transport equipment is present in most concepts. Despite their potential advantages, these designs are inappropriate for use in many public places by those with hearing problems. Because they necessitate too much equipment, these designs also limit accessibility. This project develops a more approachable deployment process for Internet or mobile apps. The only tool required for this project is a camera. To train a neural network, the frames from the video sequence must be gathered. All of the unique gestures must be captured in the collection of photos. After the frames have been recorded, a file that may communicate the gesture to the network must be created. Training and testing-related files and images are separated into two separate directories. How the network will distinguish brand-new videos that it has never seen before is properly simulated by the testing division.

I created a sign detector in this study on the identification of sign languages that can be easily expanded to recognize a wide range of new signs and hand gestures, such as the alphabet and numerals. The Python modules OpenCV, Mediapipe, Tensorflow, and Keras were used to create this project. The OpenCV feed examines the frames of real-time video from a camera in order to determine the action of a person who is being displayed at that certain moment in time. MediaPipe Holistic is used to process the video frames and extract keypoints from our hands and torso. After acquiring the relevant points, the prediction algorithm will begin producing a prediction. The hand signal is then promptly predicted by the system. The projected sign will also be visible.

Prerequisites

The prerequisite software and libraries for the sign language project are:

- Python
- IDE (Jupyter)
- Mediapipe
- Numpy
- cv2 (openCV)
- Keras
- Tensorflow

Dataset and Labelling

Signal and motion representations are contained in a dataset. Every frame that detects a gesture or motion within the defined ROI (region of interest) will be accessible via a live video feed and saved in a directory (in this case, the gesture directory). For each symbol in the collection, there are approximately 30 video clips. Each video sequence is made up of 30 frames, which are kept as Numpy arrays and each depicts a variety of noteworthy events. The motion being signed must be recognizable throughout the entire video.

Model Training

Python training is done using the machine learning platform TensorFlow. To employ transfer learning, the datasets and label files must be transformed into a TensorFlow-compatible format. The tf -record files are created using the training and testing data directories. To begin training a network using transfer learning, the object detection models must be downloaded. To reflect the dataset's class count, the configuration files must be significantly altered. To get great accuracy, 2000 training steps are necessary. Tensorflow and Keras work together to create an LSTM model that can anticipate action on the screen, in this example a motion in Sign Language. Below Figures 1 and 2 show the model accuracy and loss.

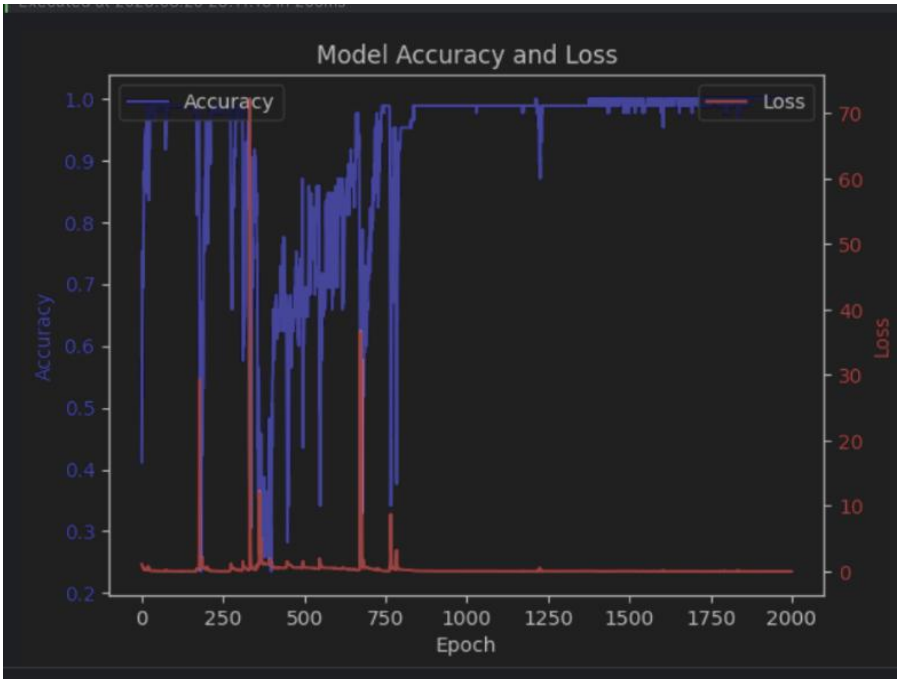


Fig 3.1: Shows the accuracy and loss at each epoch.

Project Management

- Gantt Chart: Use project management solutions that include Gantt charts to visualize and manage tasks. Gantt charts aid in task planning and scheduling. So, I can easily plan and make a Gantt chart and follow it for my future work.
- Collaboration and Communication Platforms: Use collaboration tools such as Microsoft Teams to facilitate communication among team members. These technologies allow for real-time conversations, file sharing, and quick project status updates. These tools help me to connect with my supervisor or my fellow team members in case to clear or solve my doubts.
- Documentation Tools: Use documentation tools like MS Word or Google Docs to create and manage detailed project documentation like project plans, requirements, architecture, and user guides. Making proper documentation helps me keep track of my project and each milestone of the project.
- Implementation of the Code: Tools like Colab, Jupyter can help us to implement the code. These tools can help us in several ways throughout the coding; some of them are model training, data analysis and visualization, model evaluation, etc.
- Version control system (VCS)- Git: Git can be used to keep track of code changes, manage project versions, and improve team collaboration. VCS ensures that changes are well-documented and allows you to revert to previous versions if necessary.

4. Implementation

Data collection and Pre-processing:

The initial step in using the sign language recognition system is to gather a dataset of sign language movements. This dataset consists of videos of individuals making various signs. Then, relevant properties for the LSTM model training are extracted from each video through analysis.

Data Collection: I collected a wide range of sign language gestures from the deaf population, including several common signs. Numerous recordings of each sign were made under various lighting conditions and viewing angles to ensure accuracy.

Pre-Processing: The obtained films were pre-processed to remove body keypoints using the MediaPipe software. Pre-trained hand landmark identification and pose estimation models are included in this collection. One of the keypoints that aids in our comprehension of the nuances of sign motions is the positions and orientations of the body parts.

Model Development:

This system's foundation is the development of an LSTM-based deep learning action recognition model. Since LSTM networks are excellent at processing sequential data, they can be used to capture the temporal patterns in sign language gestures.

Model Architecture: This model is made up of layers that are highly coupled and multiple LSTM layers. The LSTM layers learn the temporal correlations in the input sequences of keypoints while the dense layers partition the recognized motions into specified categories.

Model Training: From the dataset, a training set and a testing set were produced. In order to train the model using the training data, the categorical cross-entropy loss function and Adam optimizer were applied. I used TensorBoard to track the training process and show the loss and accuracy statistics.

Real-time detection and output Interpretation

After the model had been trained, I added it to a pipeline for processing real-time video feeds. The following steps were engaged in this pipeline:

Video Capture: The OpenCV framework is used to capture webcam video frames in real time.

Extraction of Keypoints: Each frame was put through the MediaPipe Holistic model to extract location and hand landmarks. The collected keypoints were then provided as input to the trained LSTM model.

Gesture Recognition: The LSTM model predicted the gesture based on the input keypoints. The model generated the probabilities for each kind of gesture.

Threshold-based decision: After a gesture was recognized, I created a criterion to determine whether it was sufficiently assured. When the maximum likelihood of that gesture is higher than the threshold, we can consider the gesture to be acknowledged.
Output Presentation: The screen displayed text for gestures that were detected as output.

System Design

Data Flow Diagram

The system may conduct input data, different processing operations on this data, and show the system to generate output data across the entire system thanks to this simple graphical formalism. It shows the inputs and outputs of the data processing and illustrates the information flow for any system or process. Rectangles, circles, and arrows are used as segmentation symbols to represent the data input, output, storage sites, and paths inside each target. A new system or its model can be examined using them. A DFD can frequently "say" things graphically that are difficult to explain in words and is useful for both technical and non-technical applications. There are four components to DFD:

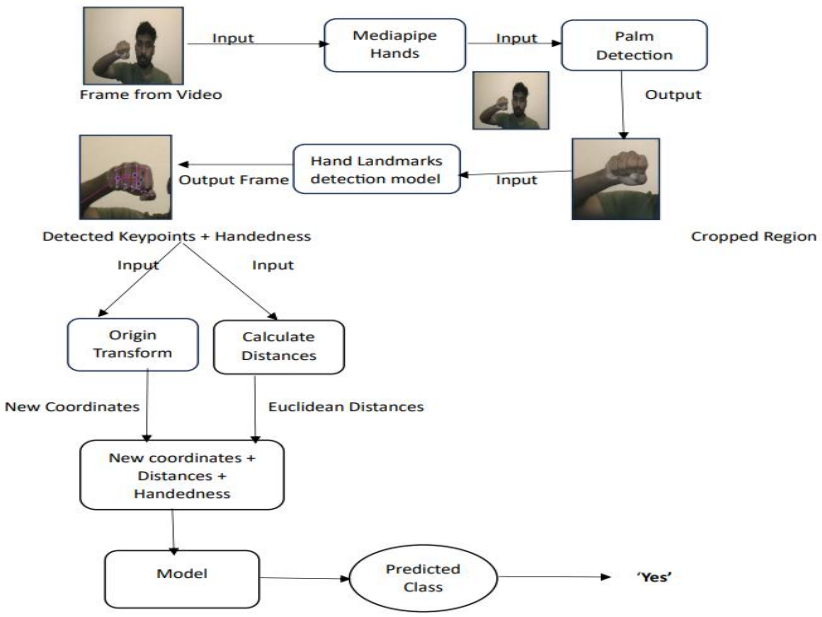


Fig 4.1: Workflow of the project in real-time

State Chart

A state chart diagram describes a state machine that exemplifies class behaviour. It shows the actual state changes rather than the processes or commands that caused them by replicating the life cycle of objects from each class. It illustrates how a thing changes from one state to another. There are mainly two states in the State Chart Diagram: The second condition is the starting point. Here are some examples of state chart diagram components:

- State: A situation in which an object is in its life cycle and meets a certain condition, performs an action, or waits for an occurrence.
- Transition: A "transition" between two states illustrates how an object in the first state acts before transitioning to the second state or event.
- Event: An event is a description of a significant occurrence that occurs at a certain time and location.

The following is the state diagram of this model.

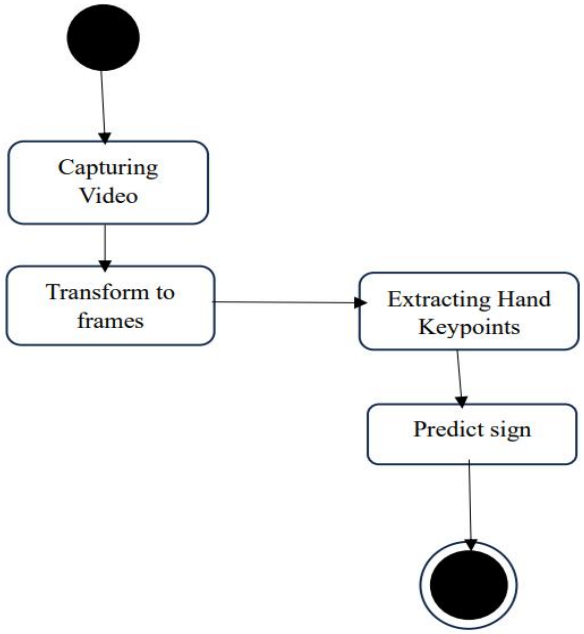


Figure 4.2 State Diagram

Use Case Diagram

Use cases are used during requirement elicitation and analysis to explain the functionality of the system. A system function that gives an actor an obvious consequence is specified by a use case. The identification of actors and use cases leads to the creation of the system boundary, which distinguishes the tasks performed by the system from those performed by its environment.

The actors are outside the limits of the system, although the use cases are inside. In a use case, the actor's point of view is used to explain the system's behaviour. It describes how the system works as a series of events that clearly affect the actor.

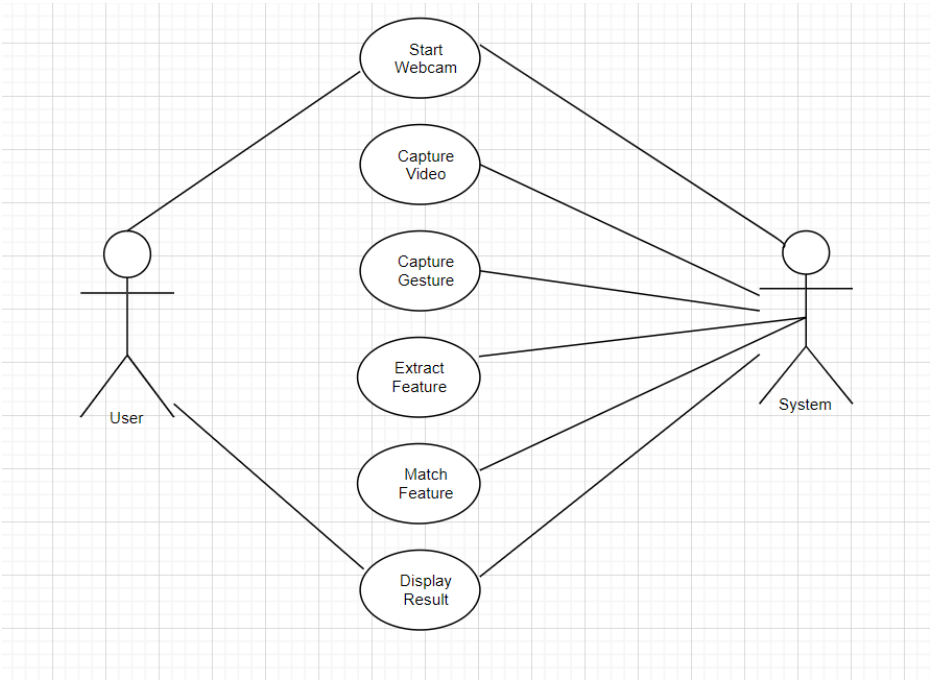


Fig 4.3 Use Case Diagram

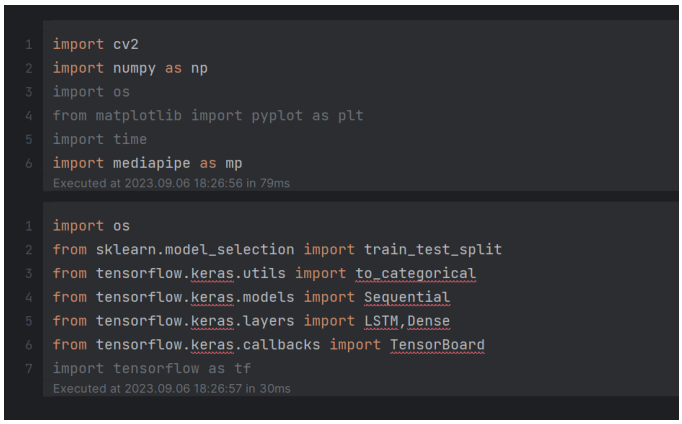
Difficulties I faced while doing this project

Limited dataset: Finding a dataset that contains several sign language gestures was very difficult. When I started the project, I searched for datasets all over the internet and websites like “Kaggle” etc. but I couldn’t find one that was perfect for my project. So, I decided to make my own dataset and I converted each video clip into 30 frames which contains 3 sign language gestures.

Latency: There is always a delay in real-time testing of my model. Sometimes the predictions of the gesture may be a little slower.

Code Snippets

I. Importing necessary libraries: The code starts by importing several Python libraries and modules. These libraries are used for various purposes in this code.



```
import cv2
import numpy as np
import os
from matplotlib import pyplot as plt
import time
import mediapipe as mp
Executed at 2023.09.06 18:26:56 in 79ms

import os
from sklearn.model_selection import train_test_split
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM,Dense
from tensorflow.keras.callbacks import TensorBoard
import tensorflow as tf
Executed at 2023.09.06 18:26:57 in 30ms
```

Fig 4.4

II. Making Keypoints

The code below uses OpenCV and the Python MediaPipe module to show landmarks and recognize gestures. For full gesture identification, the MediaPipe Holistic model is imported, and sketching tools are provided for identifying landmarks. The detection_mp function transforms colour spaces, hunts down landmarks, and identifies poses before converting the images it receives back. To aesthetically represent recognizable landmarks in photos, such as stance, left hand, and right hand, use the draw_styled_landmarks function. This code is necessary for real-time gesture recognition and landmark visualization.

```python
holistic_mp = mp.solutions.holistic # Holistic model
drawing = mp.solutions.drawing_utils # Drawing utilities
```
Executed at 2023.09.06 18:26:59 in 62ms

```python
def detection_mp(image,model):
    image = cv2.cvtColor(image,cv2.COLOR_BGR2RGB)
    image.flags.writeable = False
    results = model.process(image)
    image.flags.writeable = True
    image = cv2.cvtColor(image,cv2.COLOR_RGB2BGR)
    return image,results
```
Executed at 2023.09.06 18:27:00 in 67ms

Fig 4.5

```python
def draw_styled_landmarks(image,results):
    drawing.draw_landmarks(image,results.pose_landmarks, holistic_mp.POSE_CONNECTIONS,
                           drawing.DrawingSpec(color=(80,22,10), thickness=2, circle_radius=4),
                           drawing.DrawingSpec(color=(80,44,121), thickness=2, circle_radius=2)
                           )

    drawing.draw_landmarks(image,results.left_hand_landmarks, holistic_mp.HAND_CONNECTIONS,
                           drawing.DrawingSpec(color=(121,22,76), thickness=1, circle_radius=4),
                           drawing.DrawingSpec(color=(121,44,250), thickness=1, circle_radius=2)
                           )

    drawing.draw_landmarks(image,results.right_hand_landmarks, holistic_mp.HAND_CONNECTIONS,
                           drawing.DrawingSpec(color=(245,117
                           drawing.DrawingSpec(color=(245,66,
                           )
```
Module **mediapipe.python.solutions.holistic**

MediaPipe Holistic.

Assigned to: **holistic_mp**

Executed at 2023.09.06 18:27:01 in 31ms

Fig 4.6

III. Extracting Keypoints

The keypoints_extract function processes the landmark data from the MediaPipe comprehensive model. It collects the x, y, and z coordinates as well as the visibility information for the stance (body), left-hand, and right-hand landmarks. These values are flattened into separate arrays to allow interchange with various downstream applications, such as sign language identification. The output is a concatenated array that includes landmarks from the stance, left hand, and right hand in order to aid in further analysis and machine learning tasks.

```python
def keypoints_extract(results):
    pose = np.array([[res.x, res.y, res.z, res.visibility] for res in results.pose_landmarks.landmark]).flatten() if
      results.pose_landmarks else np.zeros(33*4)
    lh = np.array([[res.x, res.y, res.z] for res in results.left_hand_landmarks.landmark]).flatten() if results
     .left_hand_landmarks else np.zeros(21*3)
    rh = np.array([[res.x, res.y, res.z] for res in results.right_hand_landmarks.landmark]).flatten() if results
     .right_hand_landmarks else np.zeros(21*3)
    return np.concatenate([lh,rh,pose])
```
Executed at 2023.09.06 18:27:03 in 24ms

Fig 4.7

IV. Creating folders for the dataset

In the code below, we are setting up a directory structure for our dataset. The three acts we have defined are hello, thanks, and yes. We intend to capture 30 films, each with 30 frames, for each of these acts. By iterating through each action and sequence (video) number, the code tries to create a corresponding directory within the 'Data_set' folder to retain the captured data. Thanks to this methodical structuring, our dataset is kept beautifully ordered for training and analytical reasons.

Fig 4.8

V. Collecting Keypoint values for testing and training

This code captures video frames from the default camera, which is often a webcam, using OpenCV (cv2). It makes use of the MediaPipe holistic model (holistic_mp) to locate and follow holistic landmarks, such as posture and hand landmarks. The programme loops over the predefined actions and sequences, collecting a predetermined number of frames for each action-sequence combination. The process adds stylistic markers to the captured frames as it goes along and provides feedback on how effectively the data collection is progressing. The landmarks that represent important body and hand positions are extracted and saved as NumPy arrays in a directory structure. By using the 'q' key, the code can be terminated once it outputs the screen with the frames it has captured. This process aids in collecting tagged data for the training of the sign language detection model.

```
cap = cv2.VideoCapture(0)

#Mediapipe Model
with holistic_mp.Holistic(min_detection_confidence=0.5, min_tracking_confidence=0.5) as holistic:

    #Loop through Actions
    for action in actions:
        #Loop through Videos
        for sequence in range(no_sequences):
            #Loop through video length aka sequence length
            for frame_num in range(sequence_length):


                #Read Feed
                ret, frame = cap.read()

                #Make detections
                image,results = detection_mp(frame,holistic)

                #Draw Styled Landmarks
                draw_styled_landmarks(image,results)
```

Fig 4.9

```
    draw_styled_landmarks(image,results)

    #Wait Logic
    if frame_num==0:
        cv2.putText(image,'Starting Collection',(120,200),
                    cv2.FONT_HERSHEY_SIMPLEX,1,(0,255,0),4,cv2.LINE_AA)
        cv2.putText(image,'Collecting frames for {} Video Number {}'.format(action,sequence),(15,12),
                    cv2.FONT_HERSHEY_SIMPLEX,0.5,(0,0,255),1,cv2.LINE_AA)
        cv2.waitKey(2000)

    else:
        cv2.putText(image,'Collecting frames for {} Video Number {}'.format(action,sequence),(15,12),
                    cv2.FONT_HERSHEY_SIMPLEX,0.5,(0,0,255),1,cv2.LINE_AA)

    #NEW Export keypoints
    keypoints = keypoints_extract(results)
    npy_path = os.path.join(DATA_PATH,action,str(sequence), str(frame_num))
    np.save(npy_path,keypoints)

    #Show to Screen
    cv2.imshow('OpenCV feed', image)

    #Breaking the Feed
```

Fig 4.10

VI. Creating Labels and features

This code creates a label_map dictionary with unique number labels for each action category using a dictionary comprehension. Then, sequence and label empty lists are initialised. For each action and sequence combination, it cycles over the saved frames, loads the previously saved NumPy arrays containing landmark data, and adds them to the window list. This window shows the succession of important occurrences for a certain action-sequence combination in time. These sequences and the labels that go with them make up the dataset used to train and test the sign language detection algorithm. This method prepares the data for machine learning by connecting each sequence of landmarks with the appropriate action label.

```
label_map = {label:num for num, label in enumerate(actions)}
Executed at 2023.09.06 18:33:38 in 38ms

label_map
Executed at 2023.09.06 18:33:38 in 39ms

 {'hello': 0, 'thanks': 1, 'yes': 2}


sequences, labels = [], []
for action in actions:
    for sequence in range(no_sequences):
        window = []
        for frame_num in range(sequence_length):
            res = np.load(os.path.join(DATA_PATH,action, str(sequence),"{}.npy".format(frame_num)))
            window.append(res)
        sequences.append(window)
        labels.append(label_map[action])
Executed at 2023.09.06 18:34:18 in 38s 647ms
```

Fig 4.11

VII. Building and Training LSTM Neural Network

This Keras-based code defines a sequential deep learning model. An LSTM-based neural network architecture is used to identify sign language.

```
model = Sequential()
model.add(LSTM(64, return_sequences=True, activation='relu', input_shape=(30,258)))
model.add(LSTM(128, return_sequences=True, activation='relu'))
model.add(LSTM(64, return_sequences=False, activation='relu'))
model.add(Dense(64, activation='relu'))
model.add(Dense(32, activation='relu'))
model.add(Dense(actions.shape[0], activation='softmax'))
Executed at 2023.09.06 18:34:24 in 1s 524ms
```

Fig 4.12

VIII. Fitting the model

This line of code trains the specified LSTM-based model (model) using 2000 iterations using the training data (x_train and y_train). Throughout training, the model improves its capacity to recognize sign language motions and minimise category cross-entropy loss. The callbacks input specifies that a TensorBoard callback (tb_callback) should be used to track and show training metrics while the model is being trained. Through the training process, these metrics help measure the model's effectiveness and convergence, providing analysis and potential changes to improve the model's accuracy and efficiency.

```
history = model.fit(x_train,y_train,epochs = 2000, callbacks=[tb_callback])
Executed at 2023.09.06 18:40:32 in 6m 3s 917ms
3/3 [==============================] - 0s 53ms/step - loss: 0.3406 - categorical_accuracy: 0.8706
Epoch 7/2000
3/3 [==============================] - 0s 50ms/step - loss: 0.3946 - categorical_accuracy: 0.8235
Epoch 8/2000
3/3 [==============================] - 0s 50ms/step - loss: 0.7486 - categorical_accuracy: 0.7412
Epoch 9/2000
3/3 [==============================] - 0s 50ms/step - loss: 0.5015 - categorical_accuracy: 0.7294
Epoch 10/2000
```

Fig 4.13

IX. Model Summary

The below figure is the model summary which shows all the layers inside the model.

```
Model: "sequential_1"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 lstm_3 (LSTM)               (None, 30, 64)            82688

 lstm_4 (LSTM)               (None, 30, 128)           98816

 lstm_5 (LSTM)               (None, 64)                49408

 dense_3 (Dense)             (None, 64)                4160

 dense_4 (Dense)             (None, 32)                2080

 dense_5 (Dense)             (None, 3)                 99

=================================================================
Total params: 237251 (926.76 KB)
Trainable params: 237251 (926.76 KB)
Non-trainable params: 0 (0.00 Byte)
```

Fig 4.14

X. Making Prediction

After building the model we have to make predictions. In the below figure, we can see that all the predictions are true.

```
res = model.predict(x_test)
```

Executed at 2023.09.06 18:40:37 in 473ms

```
1/1 [==============================] - 0s 332ms/step
```

```
print(actions[np.argmax(res[1])])
```

Executed at 2023.09.06 18:40:38 in 14ms

```
hello
```

Add Code Cell | Add Markdown Cell

```
print(actions[np.argmax(y_test[1])])
```

Executed at 2023.09.06 18:40:39 in 25ms

```
hello
```

Fig 4.15

XI. Real time Testing

The final stage is the real time testing.

```
#New Detection Variables
sequence = []
sentence = []
threshold = .4

cap = cv2.VideoCapture(0)
#Mediapipe Model
with holistic_mp.Holistic(min_detection_confidence=0.5, min_tracking_confidence=0.5) as holistic:
    while cap.isOpened():

        #Read Feed
        ret, frame = cap.read()

        #Make detections
        image,results = detection_mp(frame,holistic)

        #Draw Styled Landmarks
        draw_styled_landmarks(image,results)

        #Prediciton Logic
        keypoints = keypoints_extract(results)
        sequence.insert(0,keypoints)
```

Fig 4.16

```
        if len(sequence) == 30:
            res = model.predict(np.expand_dims(sequence,axis=0))[0]

        #Visualization
        if res[np.argmax(res)] > threshold:
            if len(sentence) > 0:
                if actions[np.argmax(res)] != sentence[-1]:
                    sentence.append(actions[np.argmax(res)])
            else:
                sentence.append(actions[np.argmax(res)])

        if len(sentence)>5:
            sentence = sentence[-5:]


        #Viz probability
        image = prob_viz(res,actions,image,colors)


        cv2.rectangle(image,(0,0),(640,40),(245,117,16),-1)
        cv2.putText(image, ' '.join(sentence),(3,30),
                    cv2.FONT_HERSHEY_SIMPLEX, 1,(255,255,255),2,cv2.LINE_AA)
```

Fig 4.17

5. Results

The goal of this project was to forecast signals using deep neural networks, Mediapipe Holistic, and forearm, hand, and finger kinematics models. With 95.4 percent accuracy on the test sets, the Mediapipe LSTM with data augmentation delivered the best results. In addition to understanding signals, this sign language detector will be able to identify and pick up on hand and produce coordinators. All of the signs will receive real-time updates. Also, I have applied a probability visualization of the sign languages in the output window which will either decrease or increase depending on the sign language we show.

I have gained more knowledge in this domain, its complexities, and the challenges faced by the sign language community.

Things I have learned during the execution of my project.

i. I have learned the importance of data pre-processing in handling real-world data.

ii. Also, I learned about extracting features for processing image and video data and also understood how to convert visual information into numerical data that can fed into the LSTM model.

iii. I have learned time management skills as I balance data gathering, preprocessing, model creation, and evaluation.

iv. Finally, I have gained experience in training LSTM models, monitoring the training progress, and evaluating their performance using appropriate metrics.

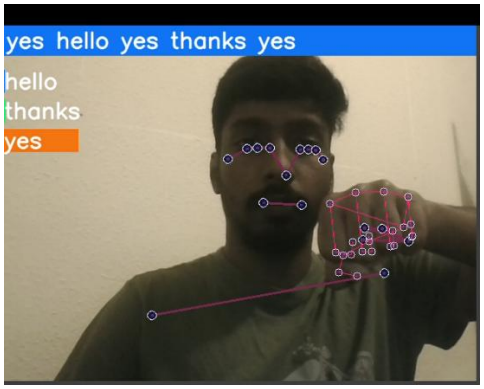


Fig 5.1. Detecting Sign Language for Yes

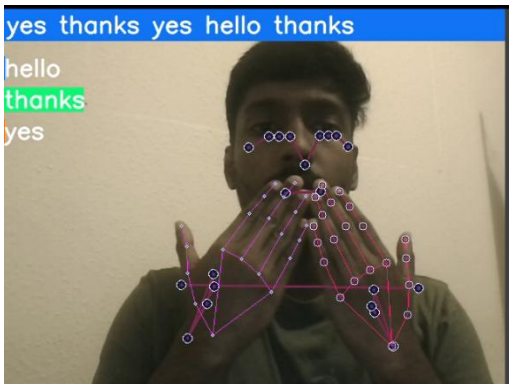


Fig 5.2. Detecting Sign Language for Thank You

5.1 EVALUATION

I’m going to evaluate my project aims and artefact in the following ways:

1. Accuracy and Recognition Performance: One of my project's primary goals is accurate sign language gesture recognition. The classification accuracy on a test dataset, including precision, recall, and F1-score, can be evaluated to evaluate the model's capacity to reliably recognize varied sign language movements.
2. Real-time Performance: Real-time gesture recognition is a key project aim, the system's speed and responsiveness can be evaluated. To ensure that the system provides users with instant feedback, the frame rate and inference time on the camera stream can be measured.
3. Cross-Validation and Generalisation: The dataset can be divided into many folds for cross-validation in order to test the model's generalization capabilities. This ensures that the model performs well on previously unknown data and does not overfit the training set.
4. Comparative Analysis: A comparison with other cutting-edge sign language detection systems should be undertaken if possible. This comparison can shed light on the system's competitiveness and advantages over existing approaches.
5. Error Analysis and Improvements: Analysing misclassified gestures and areas of low recognition accuracy can lead to model and data collection improvements. The performance of the model can be improved by fine-tuning it based on error analysis.
6. Documentation and Code Review: The quality and comprehensiveness of documentation, as well as code review, are significant evaluation factors. Because of the well-documented code and clear explanations, future developers will be able to easily understand and manage the project.
7. Project Success Metrics: Finally, the project's success can be measured against predefined success criteria such as achieving the desired accuracy and meeting real-time performance requirements.

The project’s aims and the artefact can be comprehensively assessed for their efficiency, accuracy, usability, and impact on accessibility and inclusivity by undertaking rigorous evaluation utilizing the following metrics and methodologies. The evaluation findings can be used to make additional adjustments and guarantee that the project meets its objectives

5.2 Related Work

Previous attempts to develop sign language detection models to predict the most accurate model [6] for commercial use have mainly used static images as their dataset. In this paper [7], we intend to predict the action in real time with a few phrases A survey of the literature on sign language detection and neural networks is shown in Table I. It presents a tabular analysis of earlier studies in this field, along with information on their methods, benefits, and drawbacks.

Paper	Methodology	Merits	Demerits
Wadhawan, A. and Kumar, P., 2020. Deep learning-based sign language recognition system for static signs. pp.7957-7968.	CNN + ReLu and Max Pooling	Outperformed existing ISL systems, High accuracy.	Cannot recognize dynamic signs, and will require a video dataset.
B. Bauer and H. Hienz, "Relevant features for video-based continuous sign language recognition, Conference on Automatic Face and Gesture Recognition "Proceedings Fourth IEEE International C. (Cat. No. PR00580), 2000, pp. 440-445, doi: 10.1109/AFGR.2000.840672.	Hidden Markov Model	Used video detection for sign language recognition.	Very old model.

Table 3. Comparison with other models

Ankita Wadhawan and Prateek Kumar used CNN to train the model on frames of images for our foundation study [8], and different optimizers including SGD and Adam were used to assess its performance. The study by B. Bauer and H. Hienz [9] uses the Hidden Markov model for continuous video detection even though it's an obsolete method. The main innovation in this proposed effort is that we will foresee what activity is being exhibited at any given time by using movies (a number of frames).

6. Conclusion

In summary, the goal of this study was to create a sign language detection system based on action recognition and LSTM networks. I successfully overcame the difficulties provided by deciphering and comprehending the choices made by this model throughout the duration of this research, making a contribution to the disciplines of computer vision, deep learning, and human-computer interaction.

In order to accurately recognize sign language gestures, I first gathered a large dataset of sign language gestures, incorporating both spatial and temporal information. I developed a strong model that can capture complex temporal correlations inside sequences of hand movements by utilizing the strength of LSTM networks. The LSTM model showed that it could comprehend and identify the changing sign language patterns, resulting in a high degree of accuracy in real-time sign language identification.

One of the most significant achievements of this research was the deployment of methods for model interpretability. Understanding how the LSTM-based model produces predictions is crucial for problem-solving and model improvement. Using visualization and interpretation tools, I revealed insights into the decision-making process, fostering transparency and confidence in this model's conclusions.

In conclusion, the demonstration of this experiment shows that LSTM-based action recognition for sign language detection has a lot of potentials. This study opens up new avenues for investigation and development in the realm of assistive technologies. By removing barriers to communication and enabling persons with hearing impairments to speak easily and inclusively, this study intends to have a positive impact on society.

6.1 Reflection

During the course of this project, I set out to develop a sign language detection and recognition system using LSTM-based action recognition. I'm pleased with my work since it has given me the opportunity to delve into the fascinating fields of computer vision, machine learning, and gesture detection.

One of the primary challenges I encountered was acquiring and preparing the dataset for the model's training. The process of gathering a wide range of sign language motions, ensuring data quality, and annotating the dataset required a lot of effort, but it was very important. The ideal architecture and hyperparameters for the LSTM model required careful testing and tuning as well.

Deciphering and presenting the model's judgments was another difficult aspect of this project. Achieving model interpretability is essential in applications like sign language recognition, where understanding how the model develops its predictions can aid in improved debugging and improvements.

One of the most enjoyable aspects of this endeavour was seeing the model's performance improve. As I gathered additional data and refined the model, I observed a noticeable improvement in accuracy as well as an increase in the system's ability to recognise sign language motions.

I see a number of areas that could be improved moving forward. Incorporating translation abilities to convert sign language into text or speech is just one of the exciting future options. Another is enhancing model interpretability through sophisticated visualization techniques. By addressing the challenges of real-time recognition and expanding the dataset to cover a greater variety of sign language variations, the project's capabilities could also be enhanced.

In conclusion, working on this project has been a fulfilling experience that has allowed me to learn more about how accessibility and technology are related. In the rapidly evolving field of machine learning, it has stressed the importance of continuing learning and flexibility. I'm happy with the progress so far and can't wait to continue expanding this system for recognizing sign language.

6.2 Future Work

Future research may go one step further and create a mobile app that can categorize whole word symbols using facial expressions and related hand gestures from the face. The app will be available for download on Apple and Android platforms.

7. References

[1] Razieh Rastgooa, Kourosh Kiania, Sergio Escalerab,, “Hand sign language recognition using multi-view hand skeleton”,

Electrical and Computer Engineering Department, Semnan University, Semnan 3513119111, Iran

Department of Mathematics and Informatics, Universitat de Barcelona and Computer Vision Center, Barcelona, Spain Received 18 September 2019, Revised 26 December 2019, Accepted 21 February 2020, Available online 22 February 2020, Version of Record 11 March 2020. https://doi.org/10.1016/j.eswa.2020.113336

[2] Rastgoo, R., Kiani, K. & Escalera, S. “Real-time isolated hand sign language recognition using deep networks and SVD.” J Ambient Intell Human Comput 13, 591–611 (2022). https://doi.org/10.1007/s12652-021-02920-8

[3] R. Bhadra and S. Kar, "Sign Language Detection from Hand Gesture Images using Deep Multi-layered Convolution Neural Network," 2021 IEEE Second International Conference on Control, Measurement and Instrumentation ,2021,pp.196-200,doi:10.1109/CMI50323.2021.9362897. https://ieeexplore.ieee.org/document/9362897

[4] “Convolutional and recurrent neural network for human activity recognition: Application on American sign language”

Hernandez V, Suzuki T, Venture G (2020) Convolutional and recurrent neural network for human activity recognition: Application on American sign language. PLOS ONE 15(2): e0228869. https://doi.org/10.1371/journal.pone.0228869

[5] Neziha Jaouedia, Noureddine Boujnahb, Med Salim Bouhlelc, “A new hybrid deep learning model for human action recognition.”,

National Engineering School, Avenue Omar Ibn El Khattab Zrig Eddakhlania, Gabes 6072, Tunisia,TSSG-Waterford Institute of Technology, West Campus Carriganore Waterford, X91 P0H, Ireland,SETIT Higher Institute of Biotechnology, Soukra Road km 4 BP 261, Sfax 3000, Tunisia

Received 9 January 2019, Revised 4 September 2019, Accepted 8 September 2019, Available online 9 September 2019, Version of Record 11 May 2020.

https://doi.org/10.1016/j.jksuci.2019.09.004

[6] H. Wang and C. Schmid, "Action recognition with improved trajectories", Proceedings of the IEEE international conference on computer vision, pp. 3551-3558, 2013.

[7] U.M. Prakash and V.G. Thamaraiselvi, "Detecting and tracking of multiple moving objects for intelligent video surveillance systems", Second International Conference on Current Trends In Engineering and Technology-ICCTET 2014, pp. 253-257, 2014, July.

[8] A. Wadhawan and P. Kumar, "Deep learning-based sign language recognition system for static signs", Neural computing and applications, vol. 32, no. 12, pp. 7957-7968, 2020.

[9] B. Bauer and H. Hienz, "Relevant features for video-based continuous sign language recognition", Proceedings Fourth IEEE International Conference on Automatic Face and Gesture Recognition (Cat. No. PR00580), pp. 440-445, 2000.